SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	**D.Hemasree**	**Reg. No.:**	**(192524114)**
Section / Batch:	**2025**	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q1. Debugging Linear Search Code (Incorrect Return Outside Logic)

Given (Buggy) Code

```
def linear_search(arr, key):
    found = False
    for i in range(len(arr)):
        if arr[i] == key:
```

```
            found = True
            pos = i
    return pos
```

Problem Identification — Analyzing the Control Flow

- The variable 'pos' is only created inside the 'if' block, when a match is found. It is never initialized before the loop starts.
- If the key does NOT exist in the array, the 'if' condition never becomes true, so 'pos' is never assigned any value.
- The 'return pos' statement, placed after the loop, then tries to return a variable that was never created — this causes an UnboundLocalError (a runtime crash) instead of a clean 'not found' result.
- The 'found' flag is calculated but never actually used to decide what to return — it is dead logic that misleads the reader about how the function reports failure.
- Even when the key IS found, the loop keeps scanning the remaining elements instead of stopping immediately, wasting comparisons and returning the LAST matching index instead of the first.

Debugging — Root Cause and Fix

- Root cause: the function relies on a variable ('pos') that only exists conditionally, and it has no defined return value for the 'not found' case.
- Fix 1: Return the index immediately (return i) the moment a match is found — this removes the need for the 'found' flag and the uninitialized 'pos' variable entirely.
- Fix 2: Place a single 'return -1' after the loop, which only executes if the loop completes without finding a match — this gives a clear, well-defined signal for the 'key not present' case.
- This restructuring ensures every code path (found / not found) has an explicit, correct return value.

Optimization — Readability and Maintainability

- Early exit: returning as soon as a match is found avoids scanning the rest of the array unnecessarily, saving comparisons in the average and best case.
- Removed the unused 'found' boolean flag — it added no real functionality and only confused the control flow.
- Used a single, well-defined sentinel return value (-1) for 'not found', which is a standard, easily understood convention.
- Added clear comments and a docstring so the function's behavior is documented and easy to maintain.

Time Complexity Analysis of the Corrected Algorithm

Best Case: O(1) — the key is found at the very first index, so the loop exits immediately.

Worst Case: O(n) — the key is at the last position or absent, so all n elements must be checked.

Average Case: O(n) — on average, about half the array is scanned before a match (or the full array if absent).

Space Complexity: O(1) — no extra data structures are used; only a loop counter is needed.

The early-return optimization does not change the worst-case asymptotic complexity (still O(n)), but it meaningfully reduces the actual number of comparisons performed whenever the key is found before the last index — improving practical, real-world performance.

Corrected and Optimized Code

```
def linear_search(arr, key):
    """
    Performs a Linear Search on 'arr' to find 'key'.
    Returns the index of the first occurrence of 'key' if found,
    otherwise returns -1.
    """
    for i in range(len(arr)):          # traverse the array once
        if arr[i] == key:              # compare each element with the key
            return i                   # return immediately when a match is found
    return -1                          # executed only if the loop completes without a match
```

Execution — Test Code

```
def linear_search(arr, key):
    """
    Performs a Linear Search on 'arr' to find 'key'.
    Returns the index of the first occurrence of 'key' if found,
    otherwise returns -1.
    """
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1



# ---- Test 1: key is present ----
data1 = [10, 20, 30, 40, 50]
key1 = 30
result1 = linear_search(data1, key1)
if result1 != -1:
    print(f"Test 1 -> Key {key1} found at index {result1}")
else:
    print(f"Test 1 -> Key {key1} not found in the array")

# ---- Test 2: key is absent ----
data2 = [10, 20, 30, 40, 50]
key2 = 99
result2 = linear_search(data2, key2)
if result2 != -1:
    print(f"Test 2 -> Key {key2} found at index {result2}")
else:
    print(f"Test 2 -> Key {key2} not found in the array")

# ---- Test 3: key is the first element (best case) ----
data3 = [7, 14, 21, 28]
key3 = 7
result3 = linear_search(data3, key3)
print(f"Test 3 -> Key {key3} found at index {result3}")
```

Output

```
Test 1 -> Key 30 found at index 2
Test 2 -> Key 99 not found in the array
Test 3 -> Key 7 found at index 0
```

Verification & Interpretation

Running the original buggy version with a key that is absent (e.g., 99) raises an 'UnboundLocalError', proving the reported bug. The corrected version handles all three cases correctly: it returns the exact index when the key is present (Test 1 and Test 3), and returns -1 with a clear message when the key is absent (Test 2) — with no crash. This confirms the debugging is correct and the function is now safe, readable, and maintainable.

Code of Conduct

I _____**D.Hemasree (192524114)** _____ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____